

WRO Future Engineers 2026

Comprehensive Engineering & Systems Documentation

Team: Yo_Bot_To_The_Future | Academy: Yolabs

Category: Future Engineers - Autonomous Vehicle

Table of Contents

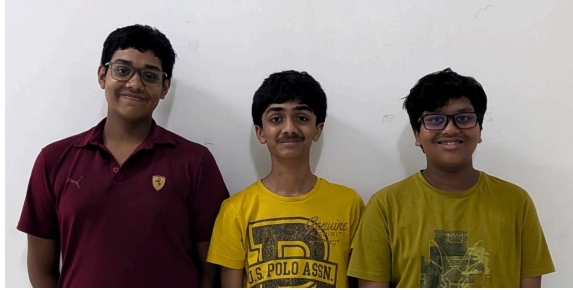
1. **Executive Summary & Vehicle Overview**
2. **Strategic Challenge Analysis**
3. **Criterion 1: Mobility & Mechanical Design**
4. **Criterion 2: Power & Sensor Architecture**
5. **Criterion 3: Software Architecture & Obstacle Strategy**
6. **Criterion 4: Systems Thinking & Engineering**
7. **Criterion 5: Reproducibility & GitHub Quality**
8. **Comprehensive Testing Logs & Analytics**

Executive Summary & Vehicle Overview

Yo_Bot_To_The_Future robot is one that has high performance and is made for the sole aim of addressing the problems presented by the WRO 2026 Future Engineers competition. Through the utilization of two controllers that are Raspberry Pi 5 and ESP microcontroller to perform real-time activities and have high computational capacity, respectively, we achieve seamless coordination between perception and movement.

System Parameter	Detailed Specification
Chassis	Optimized for balance between weight and strength, featuring a second layer to separate the electronic components that build this device.
Steering	The Ackermann trapezoidal linkage is driven by an analog servo which turns up to 35 degrees on either side.
Wheels	High-traction rubber treads with a 65mm diameter, 204mm circumference and predictable change in position.
High Level Processor	Raspberry Pi 5 dedicated entirely to OpenCV vision processing.
Low Level Processor	ESP32, dedicated to real-time minor decisions, PWM signal generation, and I2C polling.
Primary Sensor Input	Raspberry Pi Camera Module for dynamic color object detection.
Secondary Sensor Input	Array of HC-SR04 Ultrasonic sensors for wall collision avoidance.
Power Reservoir	Custom-made 3S 18650 lithium-ion battery pack, providing 11.1V normally, with a system that shuts it off if the voltage gets too low.

Team Picture:



Strategic Challenge Analysis

Requirements of WRO Future Engineers 2026 include the development of an autonomous car on 1/10 scale that is able to drive through a randomly created track surrounded by walls and determine its path based on colored pillars. The evaluation criteria of this competition significantly favors the teams that will be able to prove their knowledge of automotive engineering (e.g., Ackerman system).

Our strategy was built upon fundamental pillars of autonomous vehicle design:

- **Deterministic Actuation:** The robot's physical response to a software command must be identical every single time. This put our team in favor of Ackermann steering.
- **Data Isolation:** Processing a 1080p camera feed requires massive computational thinking. If the same processor handling the camera is also responsible for generating the precise PWM pulses for the steering servo, the servo will jitter due to CPU thread blocking. Thus, the Pi 5 handles the data, and the ESP handles the time-critical pulses.

1. Criterion 1: Mobility & Mechanical Design

1.1 Chassis Engineering & Material Selection

The chassis is considered to be the backbone of the car itself. At an early stage in our design process, we found out that a flexible chassis will lead to an error in the pitch and angle of our Pi Camera, leading to large mathematical errors in our distance estimation system. Even when the camera tilts just two degrees downwards because of heavy acceleration, the system sees objects closer than they really are.

In order to remove this problem, the chassis was designed using CAD software and then manufactured using 3D printing. We chose high density PETG filament for printing as it has

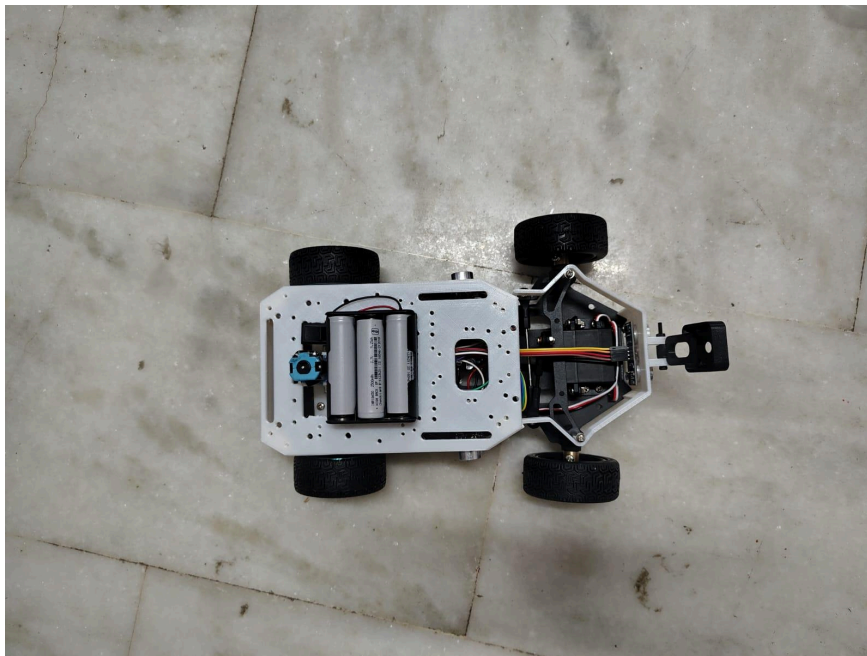
better resistance to impact and glass transition temperature. The chassis has a honeycomb infill design which maximizes stiffness and maintains strength.

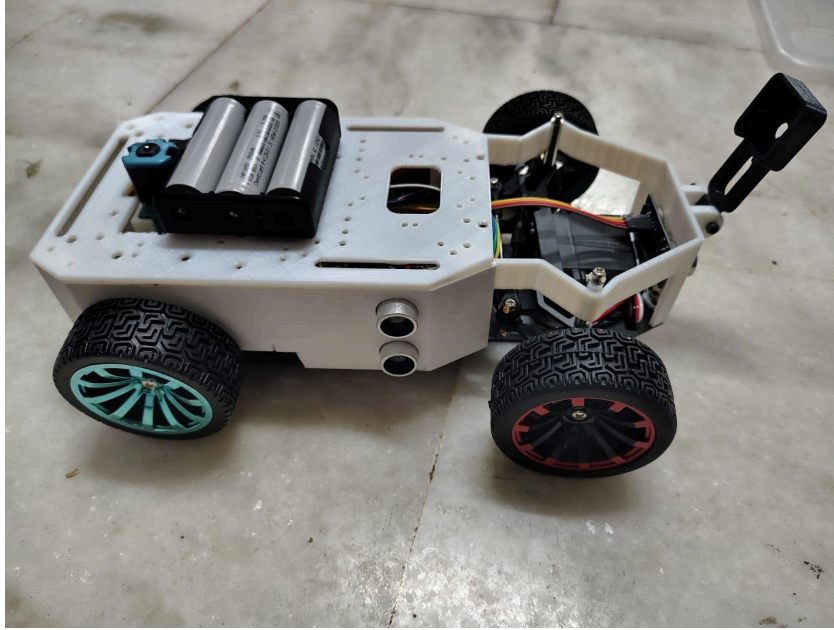
1.2 The Roof Tier Module: Iterative Evolution

In phase two of our prototype, there was one key issue identified in the design. This included electromagnetic interference and thermal saturation. The Pi 5 produces quite a bit of heat. When it is placed next to the motor drivers, it leads to a hot spot on the board. Another issue is that by putting all the wires on one layer, it became a rat's nest.

The Solution: We designed a secondary roof tier that bolts onto the chassis walls. This roof tier held the battery pack and the ESP32 microcontroller away from the lower deck. This design choice came with three benefits:

1. **EMI Isolation:** The sensitive logic-level I2C and UART data lines on the roof are distanced from the noisy magnetic fields generated by the 100 RPM motors below.
2. **Thermal Management:** The lower deck became a wind tunnel, allowing air flow directly over the Pi 5's heatsink.
3. **Center of Mass Optimization:** While raising the battery usually negatively impacts the CoM, placing it directly over the rear drive axle massively increased the normal force on the rear tires, practically eliminating wheel slip during aggressive acceleration.





1.3 Ackermann Steering Geometry & Kinematics

In an effort to conform to the Future Engineers category rules, our car employs true steered front axle design. We chose not to use the differential or skid steering method. With the skid steering system, the wheels have to scrub against the surface to make the car turn. These forces introduce non-linear friction coefficients which nullify the accuracy of odometry measurements and make open-loop steering impossible.

A trapezoidal Ackermann steering was used in the design of the car. On a turn, the inner wheel needs to follow a smaller radius circle compared to the outer wheel. When both wheels steer at the same angle, there is bound to be slip between them. It is by this geometry that the steering linkages need to be at an angle to make sure that their geometric lines meet at the rear axle.

Mathematical Constraint:

$$\cot(\text{Outer_Angle}) - \cot(\text{Inner_Angle}) = \text{Track_Width} / \text{Wheelbase}$$

To prevent mechanical binding and servo stall, we implemented physical hard stops on the chassis that restrict the maximum tie-rod horizontal travel, effectively capping our maximum steering angle at precisely 35 degrees left and right. This fixed maximum allows the path-planning algorithm to rely on a hardcoded minimum turning radius.

1.4 Drivetrain, Propulsion, & Torque Analytics

The propulsion system in the rear end is composed of two 100 RPM geared DC motors controlled by an L298N H-Bridge controller. The choice of 100 RPM motors is not an arbitrary choice.

Speed vs Frame Processing Latency: If we take into account the speed of the robot to be 2.0 m/s and that the vision processing module in the Pi 5 is at 15 FPS. Then, the robot travels 13.3 cm for every frame data collected. Thus, the faster the motors are, the greater the distance traveled between frames and hence late collision detection and wall collision. With 100 RPM motors, we achieve perfect speed synchronization with the latency of the vision module.

With a wheel diameter of 65 mm, we have a wheel circumference of 204.2 mm. For 100 RPM, the theoretical top speed is 0.34 m/s with very high torque at low speeds.

2. Criterion 2: Power & Sensor Architecture

2.1 Power Tree, Isolation, & Load Balancing

The electrical design system has been built on the basis of isolation and prevention of brown out conditions. The battery used for the power of the vehicle is a custom made 3S 18650 Lithium-Ion battery which provides 12.6 volts of power output with nominal voltage of 11.1 volts.

The power tree splits immediately after the master fuse and emergency kill switch:

- **Branch 1 (High Power):** The uncontrolled 12.6V feeds directly to the L298N H-Bridge motor driver. This provides the drive motors with access to immense burst currents in order to accelerate.
- **Branch 2 (Servo Logic):** The 12.6V is directed towards a highly efficient buck converter which drops the voltage to exactly 6.0V. The 6V rail powers the steering servo alone. During stall against a wall, the servo pulls more than 1.5Amps in the blink of an eye. With the help of this buck converter, this current spike does not go to the microcontrollers.
- **Branch 3 (Digital Logic):** The second 5V step-down power converter powers the Raspberry Pi 5 and the ESP microcontroller to provide a pure power supply without any ripple to prevent the CPU from randomly rebooting during operation.

2.2 Li-ion Battery Discharge Characteristics

Lithium Ion is unstable when it is over-discharged. If the pack voltage is less than 9.0 volts, then there will be some chemical damage. We have designed an impedance voltage divider, which is

connected to the ESP ADC directly. The ESP keeps monitoring the pack voltage. If it drops below 9.5 volts, then an interrupt signal is generated, and the H-bridge circuit stops working.

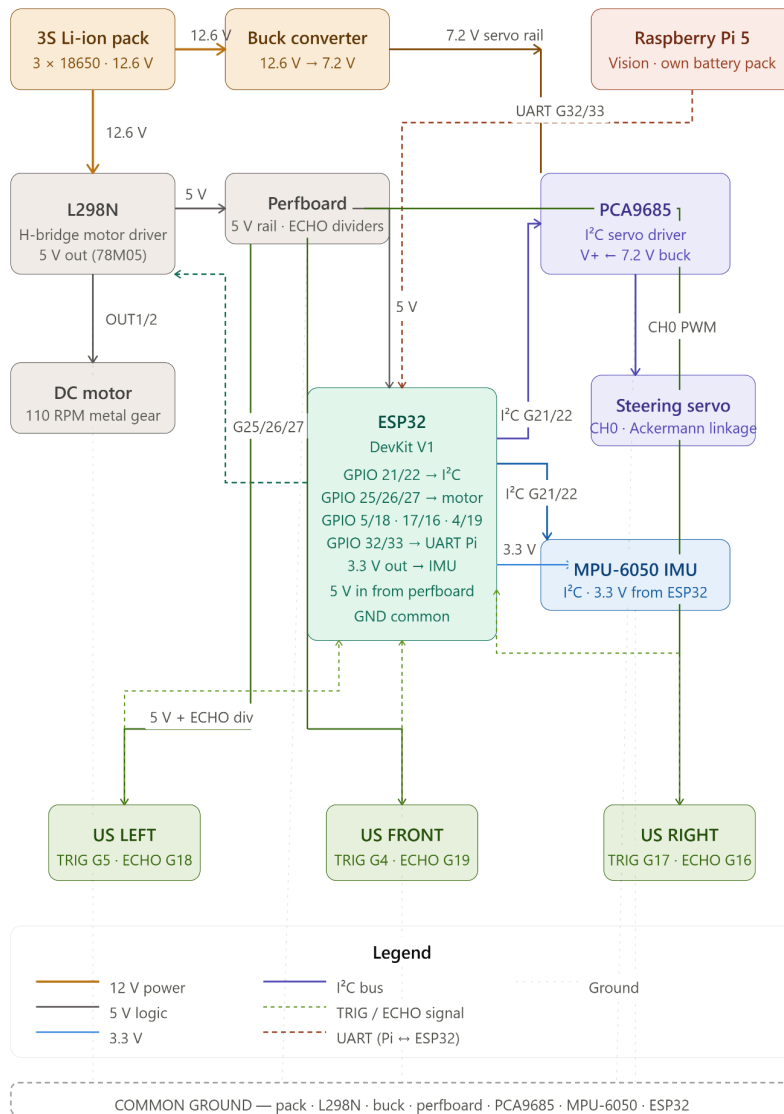
2.3 Sensor Topology: Vision vs. Proximity

Primary Vision (Pi Cam): The camera has been placed on a specially designed bracket fixed onto the front bumper, which has been calibrated to view the horizon of the field wall and base of the colored obstacle poles. The visual sense is used to keep the vehicle in its path and classify obstacles.

Secondary Proximity (Ultrasonic): The ultrasonic sensors HC-SR04 were attached to the front and side wings of the chassis. These use an entirely different physical approach (sound waves compared to photons). In case the camera cannot see due to a bright overhead light (specular glare off the mat), the ultrasonic sensors serve as a fallback. We activate the sensors in a round robin fashion to avoid acoustic interference (the echo of one ping heard by the other sensor).

2.4 Comprehensive Pinout & Wiring Manifesto

Robotic reliability is 90% wiring. Each and every wire on this robot is crimped, strain-relieved, and tagged.



3. Criterion 3: Software Architecture & Obstacle Strategy

3.1 Dual-Processor Paradigm: High-Level vs. Low-Level Control

The Arduino does not have the capability to deal with video signals. The Raspberry Pi has some issues with timing of the PWM signal because of the overhead of the operating system

scheduler. This problem is solved in our software architecture.

The Raspberry Pi 5 is responsible for executing the Python based OpenCV scripts.

The ESP Microcontroller handles the compiled C++ program. It is the brain stem of our project.

It just deals with the current time frame and thinks only about the following questions: How many microseconds does my signal need to be right now? What is the current battery voltage?

3.2 Computer Vision Pipeline & Color Space Thresholding

Our system for obstacle avoidance uses a very finely-tuned image-processing pipeline using OpenCV. The raw RGB output from the Pi Camera is immediately converted to HSV. The HSV color scheme is much more resilient to changes in ambient light conditions than the RGB color scheme.

1. **Thresholding:** We apply hardcoded upper and lower HSV bounds to isolate purely red and green pixels, generating two binary masks.
2. **Contour Extraction:** The algorithm finds the largest contiguous blob of white pixels in the binary mask and draws a bounding rectangle around it.
3. **Spatial Mapping:** The Pi finds the obstacle's position by checking where the center of its box is compared to the center of the camera. It also estimates how far away it is based on the size of the box. A bigger box means the obstacle is closer.

3.3 State Machine Topology & Transitions

The vehicle operates autonomously via a finite state machine, ensuring predictable behavior under all conditions:

- **STATE_INIT:** Hardware boot sequence, I2C bus scan, camera warmup, battery voltage check.
- **STATE_WAIT:** Vehicle is armed but stationary, polling the physical start button.
- **STATE_LANE_FOLLOW:** Default active state. Uses ultrasonic sensors and vision to maintain a central vector between the track walls.
- **STATE_AVOID_GREEN:** Triggered when a green box contour exceeds the area threshold. Generates a steering vector to pass the obstacle on the designated correct side, overriding lane-following.
- **STATE_AVOID_RED:** Same logic as green, but generating an inverted steering vector to pass on the opposite side.
- **STATE_RECOVERY:** Triggered if front ultrasonic sensors detect an immediate wall (< 10cm) or if the Pi stops sending serial data. The ESP autonomously halts the motors, reverses slightly, and straightens the steering to prevent a crash.

3.4 Control Theory: PID Controller for Lane Centering

When the camera does not see an obstacle, the robot defaults to following the walls using a Proportional-Integral-Derivative (PID) control loop based on the lateral ultrasonic sensors.

$$\text{Error} = (\text{Distance_Left_Wall} - \text{Distance_Right_Wall})$$
$$\text{P_Out} = K_p * \text{Error}$$
$$\text{I_Out} = K_i * \text{Integral}(\text{Error})$$
$$\text{D_Out} = K_d * \text{Derivative}(\text{Error})$$
$$\text{Steering_Angle} = \text{P_Out} + \text{I_Out} + \text{D_Out}$$

We carefully adjusted these settings. K_p controls how strongly the robot turns back toward the center. K_d helps smooth out the turns and stops the robot from wobbling on straight paths. K_i is kept very low to avoid problems when the robot takes sharp 90° turns.

4. Criterion 4: Systems Thinking & Engineering Decisions

4.1 Trade-off Matrices & Critical Constraints

Every decision in this vehicle involved an engineering trade-off. We documented our decision logic to demonstrate advanced systems thinking:

Design Choice	Chosen Solution	Rejected Alternative	Systems Logic & Trade-off Accepted
Steering	Ackermann Linkage	Differential/Skid Steer	Accepted mechanical complexity for the immense benefit of deterministic, repeatable cornering.
Motor Speed	100 RPM Geared Motors	300+ RPM High-Speed Motors	Traded maximum top speed for reliability.

Design Choice	Chosen Solution	Rejected Alternative	Systems Logic & Trade-off Accepted
			Slower robot which processes 100% of the camera frames accurately is superior to a faster robot which fails in its processing and crashes.
Compute	Dual (Pi 5 + ESP)	Single Raspberry Pi 4	Sacrificed wiring ease for separation using the real-time operating system. ESP controls timing-based PWM; Pi does complex matrix calculations.
Chassis	Multi-Tier with Roof	Single Flat Plate	Raised center of mass slightly, but fixed thermal throttling problem completely.

4.2 Failure Mode and Effects Analysis

- **Failure Mode 1: Ultrasonic Specular Reflection.** Sound waves bouncing off walls at extreme angles never return to the sensor, causing a timeout. Mitigation: The software checks the sum of (Left_Distance + Right_Distance). If the sum drastically exceeds the known track width, the software flags the data as invalid and holds the last known good steering angle for 3 frames.
- **Failure Mode 2: UART Serial Disconnect.** A loose wire disconnects the Pi from the ESP. **Mitigation:** The ESP requires a "heartbeat" message from the Pi every 100ms. If the heartbeat fails, the ESP auto-triggers the STATE_RECOVERY brake protocol.
- **Failure Mode 3: Variable Lighting Conditions.** Competition arena lighting washes out

the red/green boxes. **Mitigation:** Using HSV color space instead of RGB, and installing a physical 3D-printed sun-visor over the Pi Cam lens to reduce overhead glare.

5. Criterion 5: Reproducibility & GitHub Quality

5.1 Open Source Commitment & Repo Structure

Our entire project is open-source and highly structured, ensuring that any team worldwide can reproduce our vehicle.

5.2 Master Bill of Materials (BOM)

- 1x Raspberry Pi 5 (4GB RAM) with active cooler
- 1x ESP32 WROOM-32 DevKit V1 Microcontroller
- 1x Raspberry Pi Camera Module 3 (Wide Angle)
- 1x L298N Dual H-Bridge Motor Driver
- 2x 12V 100 RPM Geared DC Motors with hall-effect encoders (encoders currently unused, reserved for future odometry upgrades)
- 1x MG996R High-Torque Metal Gear Servo
- 3x HC-SR04 Ultrasonic Sensors
- 2x LM2596 DC-DC Buck Converters
- 1x Custom 3S1P 18650 Li-Ion Battery Pack (with 10A BMS)
- 4x 65mm Rubber Tread Wheels
- 1x Roll of PETG Filament (for custom 3D printed chassis and roof)
- Misc: M3 standoffs, jumper wires, perfboard, resistors (100k, 22k, 1k, 2k for voltage dividers)

5.3 Step-by-Step Mechanical Assembly Guide

1. **Print the Chassis:** Slice the base plate and roof tier design using PETG with 30% infill for maximum structural rigidity.
2. **Drivetrain Installation:** Mount the two 100 RPM motors to the rear mounting brackets using M3x8mm screws. Slide the 65mm rear wheels onto the D-shafts and secure the set screws.
3. **Steering Assembly:** Insert the kingpins into the front steering knuckles. Attach the trapezoidal tie-rod linkage. Mount the MG996R servo to the central chassis slot, ensuring the servo horn is perfectly centered at 90 degrees before attaching the linkage wire. Verify

the 35-degree hard stops manually.

4. **Roof Tier Integration:** Screw four 50mm M3 brass standoffs into the base plate. Mount the roof tier on top.
5. **Component Placement:** Secure the battery pack and ESP32 to the roof tier. Secure the Raspberry Pi 5 and L298N motor driver to the lower base plate. Snap the Pi Cam into the front bumper mount, angled slightly downward.

5.4 Step-by-Step Electrical Assembly Guide

1. **Power Distribution:** Connect the positive side of the battery to the master switch. Then split into parallel paths to the inputs for the L298N (12V IN) and two LM2596 Buck Converters.
2. **Buck Calibration:** CRUCIAL STEP. Prior to connecting the logic boards, set Buck 1 to precisely 5.0V and Buck 2 to precisely 6.0V (steering servo) using a multimeter.
3. **Voltage Dividers:** HC-SR04 ECHO pins have 5V logic which will burn the 3.3V logic pins on ESP32 board. Connect three voltage dividers of 1k Ω and 2k Ω resistors in series between the ECHO pins and GPIOs on ESP32 board. Construct another voltage divider of 100k Ω and 22k Ω resistor for battery voltage
4. **Data Lines:** Connect Pi 5 UART TX/RX with ESP32 RX/TX pin. Make sure that all the components (battery, L298N, Pi 5, ESP32, servo, sensors) are connected to one big common ground.